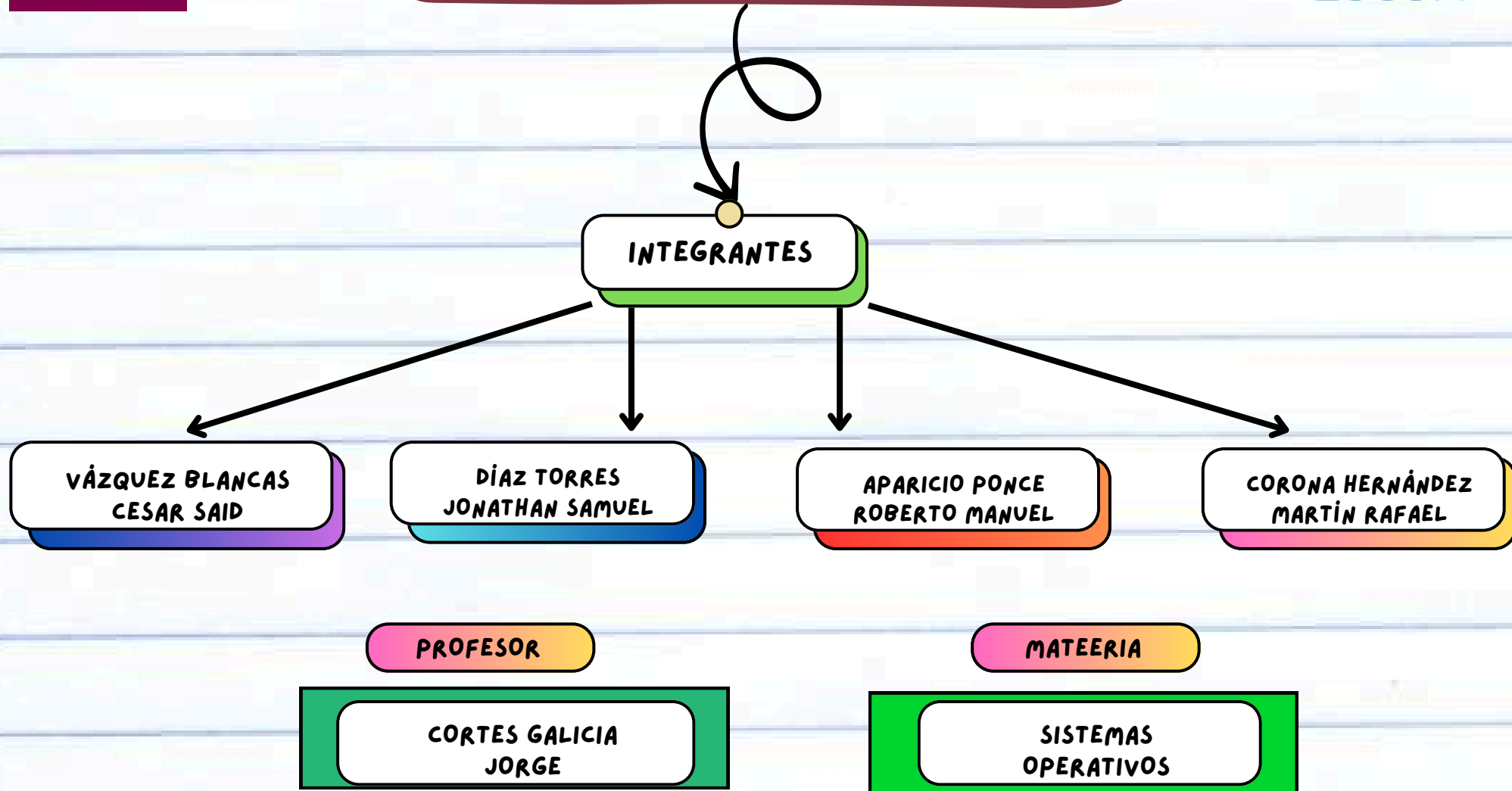




INSTITUTO POLITECNICO NACIONAL ESCUELA SUPERIOR DE COMPUTO



MEMORIA COMPARTIDA

- Memoria compartida
- Velocidad máxima
- Mejor comunicación
- no necesidad de copias adicionales

PROCESO

cooperativo: Afecta o es afectado en la ejecución de procesos y su determinación.
independiente: No afecta o se afecta por los procesos restantes

PASO DE MENSAJES

- Paso de mensajes
- Pequeñas cantidades
- Implementación mas sencilla
- Llamadas al sistema
- Canales de comunicación dedicados

COMUNICACIÓN INTERPROCESOS

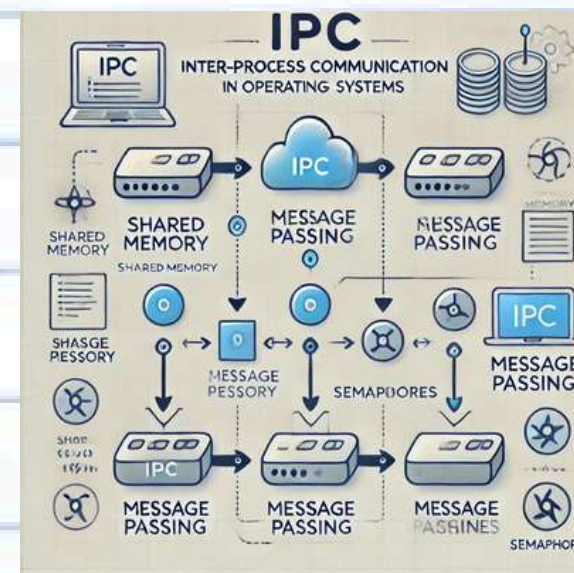
- Comunicación directa o indirecta.
- Comunicación síncrona o asíncrona.
- Almacenamiento en búfer explícito o automático

COOPERACIÓN

- Compartir información
- Acelerar cálculos
- Modularidad de procesos
- Conveniencia
- simplicidad de Comunicación

IPC

(Inter-Process Communication), es un conjunto de mecanismos que permiten a los procesos en un sistema operativo intercambiar datos y coordinarse para ejecutar tareas en común. Dado que los procesos están aislados unos de otros.



MEMORIA COMPARTIDA

- Memoria compartida
- Velocidad máxima
- Mejor comunicación
- no necesidad de copias adicionales

¿QUÉ ES?

Permite que varios procesos accedan al mismo bloque de memoria, lo que hace más rápido y fácil el intercambio de datos entre ellos.

ESPACIO COMPARTIDO PARA PROCESOS

la memoria compartida es una técnica que permite a varios programas (o procesos) acceder a la misma parte de la memoria al mismo tiempo.

MEMORIA COMPARTIDA

COOPERACIÓN

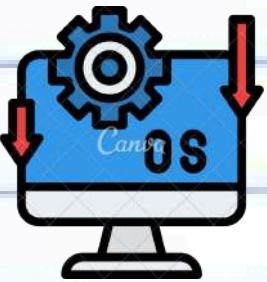
- Compartir información
- Acelerar cálculos
- Modularidad de procesos
- Conveniencia
- simplicidad de Comunicación

IPC

(Inter-Process Communication), es un conjunto de mecanismos que permiten a los procesos en un sistema operativo intercambiar datos y coordinarse para ejecutar tareas en común. Dado que los procesos están aislados unos de otros.

CONTROL DEL SISTEMA OPERATIVO

El sistema operativo define quién puede ver o modificar el bloque compartido, lo que permite mantener la seguridad y el control.



SINCRONIZACIÓN

- Envío: con bloqueo o sin bloqueo.
- Llamadas preventivas `send()` y `receive()`
- Recepción: mensajes se reciben con seguridad por el sistema

EJEMPLOS DE SISTEMAS IPC

- Windows xp
- Sistema operativo march
- Sistema Operativo UNIX/Linux
- Aplicaciones Distribuidas y Microservicios

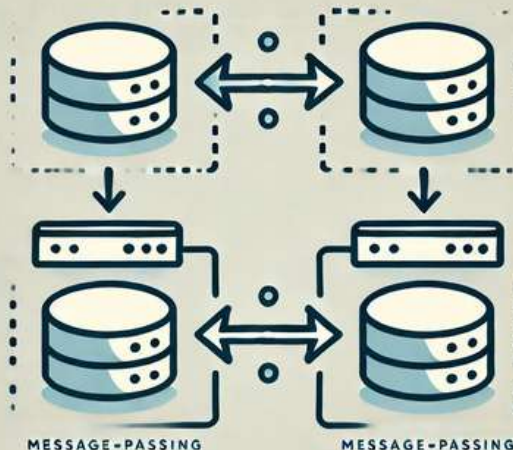
NOMBRADO

- comunicación directa: direccionamiento, simétrico y asimétrico.
- independiente: Buzones de correo o puertos.

SISTEMAS DE PASOS DE MANSAJES

ALMACENAMIENTO EN BÚFER

- Intermediario de datos: Actúa como una "espera" para los datos en tránsito.
- Evita pérdida de datos: Almacena datos temporalmente para procesarlos sin saturar la memoria.



ASLAMIENTO DE PROCESOS

Al no compartir memoria, cada proceso mantiene su propio espacio de memoria, lo cual aumenta la seguridad y reduce el riesgo de interferencia entre procesos.

TIPOS DE PUERTOS

- Puertos de conexión
- Puertos de comunicación

WINDOWS XP

- Funcionalidad
- Diseño modular
- Disminuye tiempo de implementación

LLAMADA A PROCEDIMIENTO LOCAL (LPC)

- Optimizado para Windows xp
- Establece comunicación entre procesos
- Uso de objeto puerto

PROCESOS:

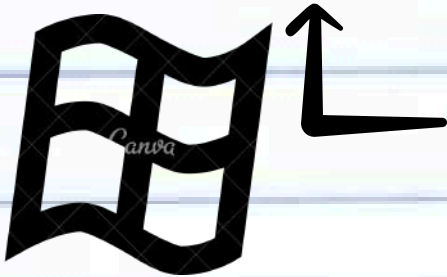
Windows xp
Comunicación en sistemas cliente-servidor

SUBSISTEMAS

- Invocación de métodos remotos de java (RMI)
- Llamadas a procedimientos remotos (RPC)
- Sockets

TÉCNICAS DE PASO DE MENSAJES

- Cola de mensajes para mensajes pequeños
- Objetos sección para mensajes grandes
- Mecanismo de retrollamada



COMUNICACIÓN EN LOS SISTEMAS CLIENTE-SERVIDOR

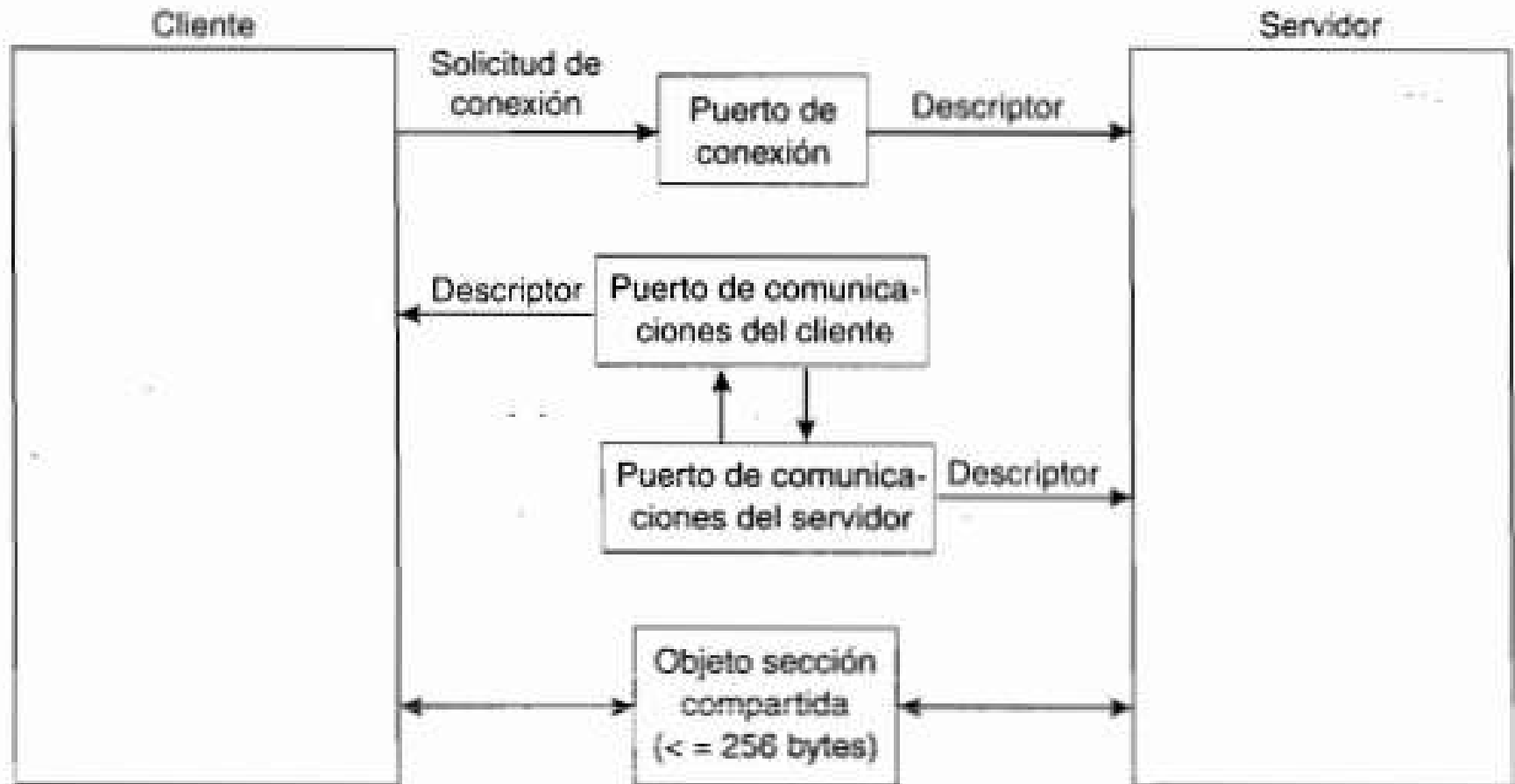


Figura 3.17 Llamadas a procedimiento locales en Windows XP.

Concurrencia



MULTIPROGRAMACIÓN

Gestión de varios procesos dentro de un sistema monoprocesador. La mayoría de los computadores personales, estaciones de trabajo, sistemas monoprocesador y sistemas operativos actuales de estas máquinas, tales como Windows y Linux dan soporte a Multiproceso Es la gestión de varios procesos dentro de un sistema multiprocesador. Recientemente, el multiproceso ha empezado a ganar espacio en los servidores y las estaciones de trabajo que demandan altas prestaciones. Windows NT es un ejemplo de un sistema operativo diseñado para estos entornos. Proceso distribuido: Es la gestión de varios procesos que 2 la multiprogramación, incorporándola para obtener sistemas monoprocesador compartidos.

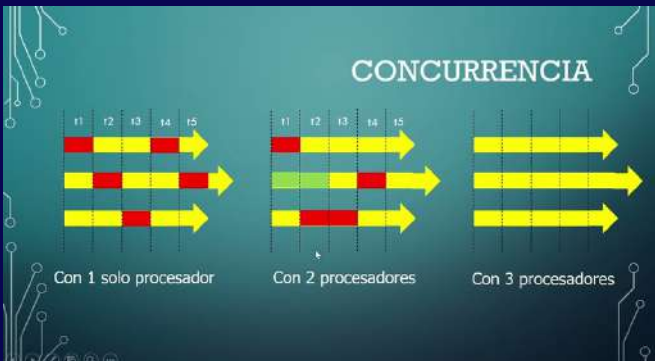


MULTIPROCESO

Es la gestión de varios procesos dentro de un sistema multiprocesador. Recientemente, el multiproceso ha empezado a ganar espacio en los servidores y las estaciones de trabajo que demandan altas prestaciones. Windows NT es un ejemplo de un sistema operativo diseñado para estos entornos.

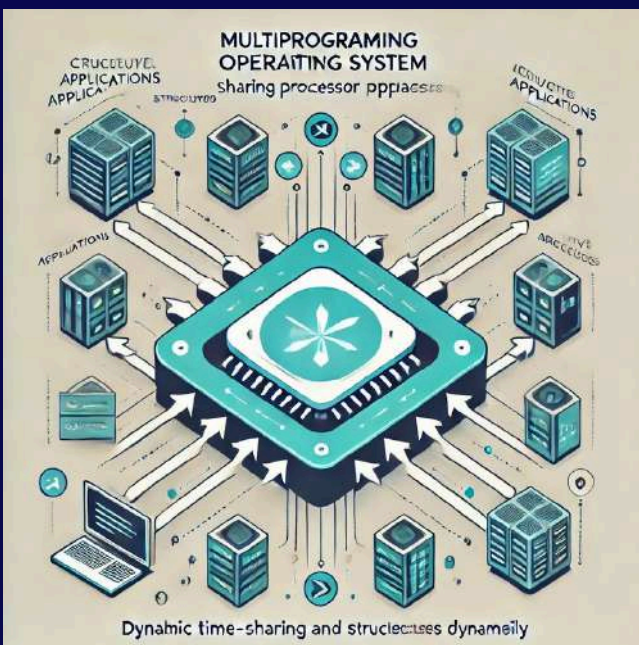
Proceso distribuido: Es la gestión de varios procesos que 2 la multiprogramación, incorporándola para obtener sistemas monoprocesador compartidos. se ejecutan en sistemas computadores múltiples y remotas.

CONCURRENCIA



Comprende la comunicación entre procesos, cómo se comparten y cómo se realiza la competencia por los recursos, sincronización de la ejecución de varios procesos y asignación del tiempo de procesador a los procesos. Estas cuestiones surgen en entornos de multiprocesadores y proceso distribuido, sistemas multiprogramados con un solo procesador.

CONTEXTOS DIFERENTES



Varias aplicaciones: La multiprogramación se creó para permitir que el tiempo de procesador de la máquina fuese compartido dinámicamente entre varios trabajos o aplicaciones activas.

Aplicaciones estructuradas: Como ampliación de los principios del diseño modular y la programación aplicaciones estructurada pueden algunas implementarse eficazmente como un conjunto de procesos concurrentes.

Estructura del sistema operativo: Las mismas ventajas de estructuración son aplicables a los programadores de sistemas y se ha comprobado que algunos sistemas operativos están implementados como un conjunto de procesos.

Transferencias de mensajes

PRIMITIVAS

```
send(destino, &mensaje);  
receive(origen, &mensaje);
```

Mediante llamadas al sistema.
SEND(destino, &mensaje): Envía un mensaje al destino especificado.
RECEIVE(origen, &mensaje): Recibe un mensaje del origen especificado, o de cualquiera si no se especifica

ASPECTOS DE DISEÑO

Evitar pérdida de mensajes: Números de secuencia-distinguir entre mensajes
Identificación de procesos.
Verificación de autenticidad.
Rendimiento.

CONCURRENCIA

Comprende la comunicación entre procesos, cómo se comparten y cómo se realiza la competencia por los recursos, sincronización de la ejecución de varios procesos y asignación del tiempo de procesador a los procesos. Estas cuestiones surgen en entornos de multiprocesadores y proceso distribuido, sistemas multiprogramados con un solo procesador.

PROBLEMA DE PRODUCTOR-CONSUMIDOR

Solución sin compartir memoria: Utiliza transferencia de mensajes. Se establece un número fijo de mensajes para mantener constante el almacenamiento. El productor envía mensajes llenos al consumidor, quien devuelve mensajes vacíos al productor

```
#define N 100 /* Número de ranuras del buffer */  
  
void producir(void) {  
    int item;  
    message m;  
    while (TRUE) {  
        produce_item(&item); /* Generar algo que poner en el buffer */  
        receive(consumer, &m); /* Esperar que llegue un mensaje vacío */  
        build_message(&m, item); /* Construir un mensaje para enviar */  
        send(consumer, &m); /* Enviar elemento al consumidor */  
    }  
}  
  
void consumer(void) {  
    int item, i;  
    message m;  
    for(i = 0; i < N; i++) /* Enviar N mensajes vacíos */  
        send(producer, &m);  
    while (TRUE) {  
        receive(producer, &m); /* Obtener mensaje que contiene elemento */  
        extract_tem(&m, &item); /* Extraer elemento del mensaje */  
        send(producer, &m); /* Devolver una respuesta vacía */  
        consume_item(item); /* Hacer algo con el elemento */  
    }  
}
```


VARIANTES



- Dirección de mensajes: Asignación de direcciones únicas a cada proceso.
- Buzones: Estructuras almacenar de datos para temporalmente mensajes. Permiten que los procesos se suspendan si el buzón está lleno.
- Cita o rendezvous: Estrategia de sincronización estricta entre emisor y receptor

COMUNICACIÓN EN UNIX



- Similar a los buzones, pero no preservan los límites de los mensajes.
- Los procesos pueden leer varios mensajes a la vez si no se gestionan correctamente.
- Se recomienda convenir en el tamaño de los mensajes o usar un carácter especial para delimitarlos

CONDICIONES DE COMPETENCIA

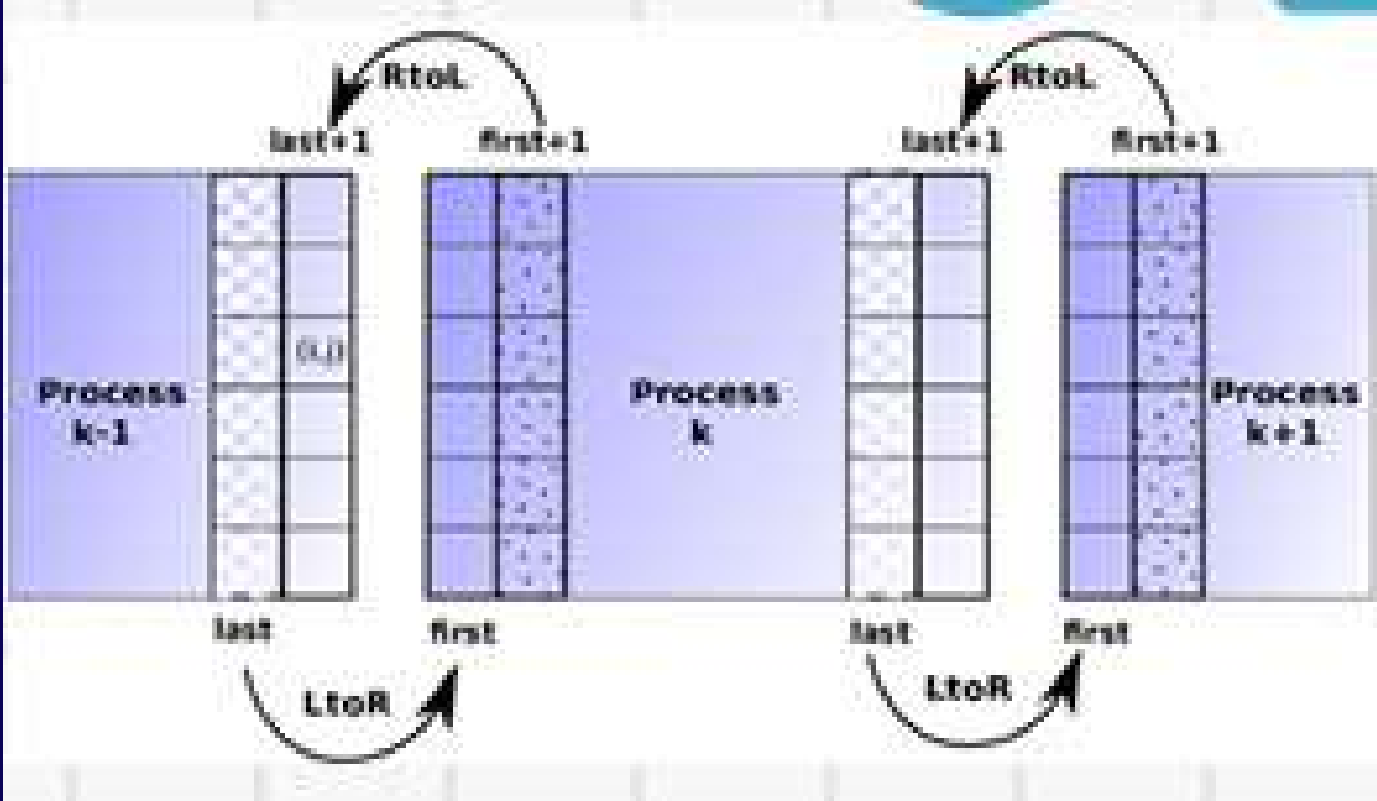
Se refiere al fenómeno que se presenta en algunos SO en dónde los procesos que cooperan para resolver tareas tienen que competir por los recursos

¿CÓMO EVITARLAS?



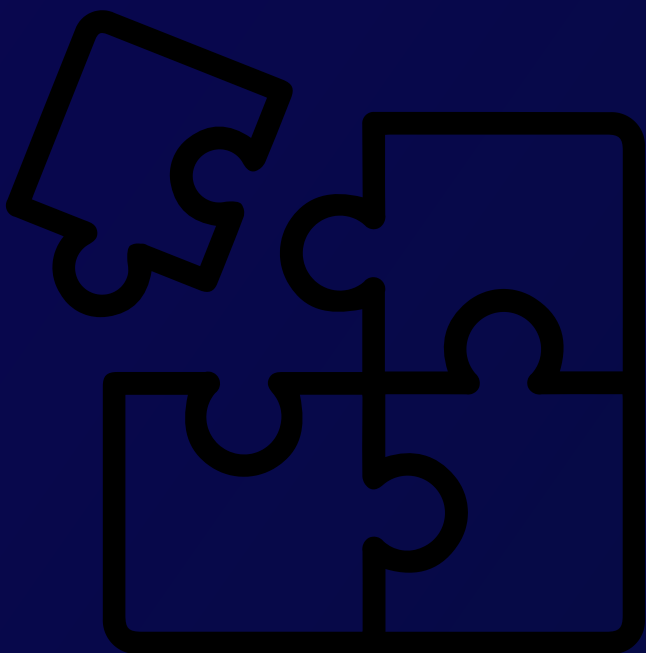
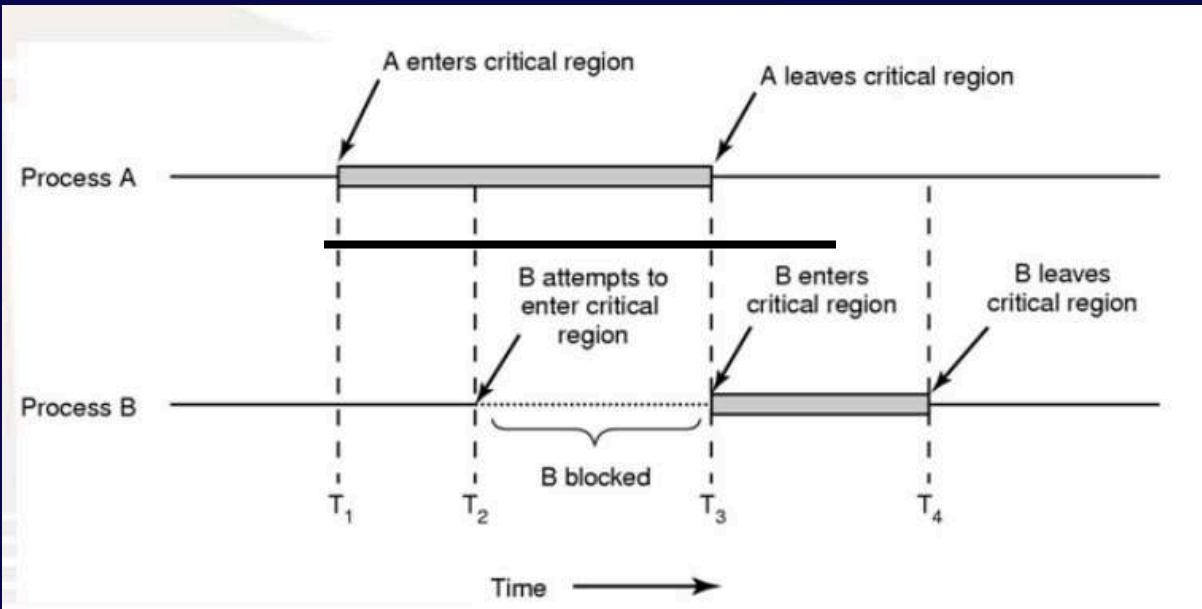
1. No debe haber dos procesos que se encuentren simultáneamente en una región crítica.
2. No se debe asumir ningún criterio respecto a las velocidades de ejecución de los procesos ni respecto al número de CPUs.
3. Ningún proceso ejecutándose en su región crítica puede bloquear a algún otro proceso. Ningún proceso puede realizar una espera indefinida dentro de su región crítica.
4. Ningún proceso puede realizar una espera indefinida dentro de su región crítica.

EXCLUSIÓN MUTUA UTILIZANDO REGIONES CRÍTICAS



EXCLUSIÓN MUTUA CON ESPERA OCUPADA

Es la propuesta de solución al problema de la región crítica. Consiste en emplear una variable que actúe como bandera para permitir el acceso de un proceso a su región crítica.



SOLUCIÓN DE PETERSON

```
Peterson ofreció esta solución para obtener exclusión mutua:
```

```
#define FALSE 0
#define TRUE 1
#define N 2 /* number of processes */

int turn; /* whose turn is it? */
int interested[N]; /* all values initially 0 (FALSE) */

void enter_region(int process) /* process is 0 or 1 */
{
    int other; /* number of the other process */

    other = 1 - process; /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process; /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

INSTRUCCIÓN TSL

Es una característica de las computadoras Intel x86, llamada instrucción TSL (Test and Set Lock) que solamente puede ejecutarse en modo núcleo, que permite entrar y salir de

```
enter_region:
    TSL REGISTER,LOCK | copy lock to register and set lock to 1
    CMP REGISTER,#0 | was lock zero?
    JNE enter_region | if it was non zero, lock was set, so loop
    RET | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0 | store a 0 in lock
    RET | return to caller
```


- Una solución simple al problema del Productor – Consumidor es utilizar semáforos:

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

/ number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */*

/ TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of empty slots */
/* increment count of full slots */*

/ infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */*

DORMIR Y DESPERTAR

- Descripción: Se refiere a la gestión de procesos que entran en estado de espera y son reactivados cuando ciertas condiciones se cumplen.
- Importancia: Evita consumos innecesarios de recursos al mantener inactivos los procesos que no pueden continuar su ejecución

- El problema del productor - consumidor bosquejado con monitores:
 - Sólo un procedimiento monitor activo en un instante de tiempo.
 - El búfer posee N zonas de memoria (*slots*)

```
monitor ProducerConsumer
condition full, empty;
integer count;
procedure insert(item: integer);
begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty)
end;
function remove: integer;
begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N - 1 then signal(full)
end;
count := 0;
end monitor;
```

```
procedure producer;
begin
    while true do
        begin
            item = produce_item;
            ProducerConsumer.insert(item)
        end
    end;
procedure consumer;
begin
    while true do
        begin
            item = ProducerConsumer.remove;
            consume_item(item)
        end
    end;
```

PRODUCER-CONSUMER EN JAVA

- Descripción: El problema del productor-consumidor es un clásico problema de concurrencia que implica la sincronización de procesos que producen y consumen datos. l
- mplementación: Se puede resolver utilizando monitores, semáforos o mutexes para controlar el acceso al buffer compartido.

PRODUCER-CONSUMER EN JAVA

- Descripción: Continuación de la solución al problema del productor consumidor en Java.
- Implementación: Se centra en garantizar la correcta sincronización entre los procesos productores y consumidores

```
public class ProducerConsumer {
    static final int N = 100; // constant giving the buffer size
    static producer p = new producer(); // instantiate a new producer thread
    static consumer c = new consumer(); // instantiate a new consumer thread
    static our_monitor mon = new our_monitor(); // instantiate a new monitor
    public static void main(String args[]) {
        p.start(); // start the producer thread
        c.start(); // start the consumer thread
    }
    static class producer extends Thread {
        public void run() { // run method contains the thread code
            int item;
            while (true) { // producer loop
                item = produce_item();
                mon.insert(item);
            }
        }
        private int produce_item() { ... } // actually produce
    }
    static class consumer extends Thread {
        public void run() { // run method contains the thread code
            int item;
            while (true) { // consumer loop
                item = mon.remove();
                consume_item(item);
            }
        }
        private void consume_item(int item) { ... } // actually consume
    }
}
```

```
static class our_monitor { // this is a monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices
    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // if the buffer is full, go to sleep
        buffer[hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N; // slot to place next item in
        count = count + 1; // one more item in the buffer now
        if (count == 1) notify(); // if consumer was sleeping, wake it up
    }
    public synchronized int remove() {
        int val;
        if (count == 0) go_to_sleep(); // if the buffer is empty, go to sleep
        val = buffer[lo]; // fetch an item from the buffer
        lo = (lo + 1) % N; // slot to fetch next item from
        count = count - 1; // one few items in the buffer
        if (count == N - 1) notify(); // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep() { try{wait();} catch(InterruptedException exc) {};}
}
```